

OBSERVAÇÕES:

- 1) Prova individual.
- 2) O valor final da avaliação será proporcional a 5,0.

1. (1,0) Considerando os princípios fundamentais de gerenciamento de memória, julgue os seguintes itens como verdadeiro (V) ou falso (F).

(f) Um programa compilado (ou executável) deve ser trazido (do disco) para a memória para que seja executado. Endereços de código compilado são vinculados (*bind*) a endereços absolutos em memória física.

(v) A memória principal e os registradores são os únicos dispositivos de armazenamento que a CPU pode acessar diretamente. O acesso a registradores é feito em um ciclo de CPU (ou menos). O acesso a memória principal pode levar alguns ciclos de CPU, causando uma paralisação (*stall*).

(v) A unidade de gerenciamento de memória é um dispositivo de hardware que mapeia endereço virtual (i.e. lógico) em endereço físico em tempo de execução. Nesse contexto, os registradores base/limite desempenham papel fundamental de modo a garantir que o espaço de endereçamento de um processo seja respeitado.

(f) Vinculação de endereço de instruções e dados a endereços de memória pode acontecer em dois estágios diferentes: em tempo de carga e em tempo de execução.

(v) A proteção de memória é necessária para assegurar a operação correta de processos em um SO. Para tanto, utiliza-se um par de registradores base e limite, os quais auxiliam a CPU no processo de verificação de cada acesso a memória gerado em espaço de usuário, de modo a se assegurar que estes sejam feitos no espaço de endereçamento adequado, definido para aquele processo de usuário.

2. (1,0) Considerando o funcionamento da unidade de gerenciamento de memória (MMU), julgue os seguintes itens como verdadeiro (V) ou falso (F).

(v) Um endereço lógico é gerado pela CPU e é traduzido em um endereço físico pela MMU. A vinculação de endereço lógico em físico em tempo de execução ocorre quando é feita uma referência a uma posição de memória.

(v) No momento em que a MMU deve traduzir um endereço lógico em físico, o registrador base (ou registrador de realocação) funciona como endereço base para o cálculo de um endereço físico e o registrador limite contém o escopo de variação de endereços lógicos. Nesse contexto, cada endereço lógico deve ser menor do que o registrador limite. Em caso de erro de escopo do registrador limite, ocorre uma *trap* por erro de endereçamento.

(v) A política de *swapping* refere-se a remoção temporária de um processo da memória principal e sua transferência para o armazenamento secundário. É desejável que os algoritmos de gerenciamento de memória levem em consideração o ciclo de vida do processo para avaliar a possibilidade de *roll out*. Por exemplo, em caso de I/O pendente (i.e. processo inicia I/O e entra na fila de processos bloqueados), é recomendável que o processo permaneça em memória, fixado de alguma forma, de modo a evitar double buffering (i.e. a perda do que foi lido em I/O e nova leitura devido ao *roll out* do processo).

(v) Critérios para a ocorrência de *swapping* podem incluir o início da operação de *roll out* se a

quantidade de memória alocada é maior do que um certo limiar (*threshold*). Além disso, caso exista a demanda por um processo que não está na memória, pode ser necessário remover um processo (*roll out*) e trazer um processo alvo (*roll in*), dependendo da disponibilidade de memória no sistema.

(v) Em caso de *swapping*, o tempo de chaveamento de contexto pode ser muito alto. Por esse motivo, em sistemas modernos, *swapping* só ocorre caso a memória livre esteja extremamente baixa.

3. (1,0) Considerando a alocação em memória contígua, selecione a alternativa correta.

(x) Em alocação por partição múltipla, devido ao tamanho da partição ser variável e a alocação de memória ocorrer quando um processo chega em uma posição que seja “grande o bastante para acomodá-lo”, com a alocação e liberação constante de partições, podem surgir buracos em memória não adjacentes (i.e. não contíguos), os quais somados poderiam acomodar um processo de tamanho m , mas isolados não são capazes de conter um processo que perfaz esse tamanho total.

() Para alocar espaço em partição múltipla, o algoritmo *worst-fit* é melhor que o *first-fit* e o *best-fit* em termos de velocidade e utilização de armazenamento.

() Quando da ocorrência de fragmentação interna, o espaço total de memória existe para satisfazer uma requisição, mas não é contíguo.

() Em fragmentação externa, a memória alocada pode ser levemente maior do que a memória requisitada; esta diferença de tamanho na memória é interno a uma partição, mas não está sendo usado.

4. (1,0) Considerando a questão referente a segmentação em gerenciamento de memória, selecione a alternativa correta.

() Segmentação consiste em um esquema de gerenciamento de memória que suporta a visão do usuário quanto a memória.

() Nesse contexto, um programa é considerado como uma coleção de segmentos. Um segmento consiste em uma unidade lógica, como por exemplo: programa principal, procedimento, função, método, objeto, variáveis locais, variáveis globais etc.

() Na arquitetura de segmentação, o endereço lógico é representado pela tupla: $\langle n.o \text{ segmento}, offset \rangle$.

(x) Todas as alternativas anteriores.

5. (1,0) Sobre paginação, julgue os seguintes itens como verdadeiro (V) ou falso (F).

(f) Espaço de endereçamento físico de um processo deve ser contíguo, alocando memória física ao processo sempre que existir memória física disponível. Com isso, evita-se fragmentação externa.

(f) Segundo essa estratégia de gerenciamento de memória, a memória física é dividida em blocos de tamanho variável chamados quadros (*frames*).

(v) Para executar um programa de tamanho N páginas, precisa encontrar N frames e carregar o programa.

(v) Ajusta uma tabela de página (*page table*) para traduzir endereços lógicos para físicos, a qual é mantida na memória principal.

(v) Proteção de memória é implementado pela associação de um bit de proteção com cada *frame*, para indicar se acesso de leitura (*read-only*) ou acesso de leitura e escrita (*read-write*) é permitido.

6. (1,0) Considere o seguinte cenário:

Tamanho de página = 2,048 bytes

Tamanho do processo = 80 Kbytes

Calcule:

a. (0,5) Quantidade de páginas para a qual o processo é mapeado.

Resposta: 40 páginas.

b. (0,5) Fragmentação interna sofrida na última página.

Resposta: 1920 bytes.

7. (1,0) Sobre as técnicas para estruturação de tabelas de páginas, julgue os seguintes itens como verdadeiro (V) ou falso (F).

(v) No modelo de paginação hierárquica, ocorre a quebra o espaço de endereçamento lógico em múltiplas tabelas de páginas (a de nível externo é fixa as outras podem sofrer *swap out*).

(f) No modelo de paginação hierárquica, a tabela de página de 1º nível é suficiente para tradução de endereço lógico em físico, não sendo necessariamente obrigatório o carregamento de tabelas de páginas de 2º nível.

(f) No modelo de paginação por *hashing*, para que o endereço lógico seja usado, primeiramente ele precisa passar por uma função *hash* que fornecerá como resultado o índice de busca na tabela *hash*. Cada entrada da tabela *hash* tem relação com uma única página, não ocorrendo colisões no resultado fornecido pela função *hash*.

(f) No modelo de paginação hierárquica, por *hash* e paginação invertida cada processo possui sua própria tabela de página, a qual mantém todas as páginas lógicas possíveis.

(f) No modelo de paginação invertida, aumenta-se a quantidade de memória necessária para armazenar cada tabela de página, mas por outro lado, diminui o tempo de busca necessário na tabela quando ocorre uma referência de página.

8. (1,0) Sobre memória virtual, julgue os seguintes itens como verdadeiro (V) ou falso (F).

(v) Memória virtual consiste na separação de memória lógica de usuário da memória física. Assim, somente parte do programa precisa estar em memória para executar.

(v) A estratégia de paginação por demanda (*demand paging*) trazer o processo à memória somente quando este for necessário. Caso seja implementada via paginador *lazy swapper*, nunca traz uma página para a memória a menos que ela seja necessária no futuro.

(f) Na estratégia de *pure demand paging*, o processo inicia com um conjunto mínimo de páginas em memória.

(f) Referência de localidade (*locality of reference*) refere-se a mudança de contexto natural sofrida por um processo durante seu tempo de vida. Um processo internamente, pode, por exemplo, chavear entre funções, onde cada função ao ser carregada na CPU possui sua “localidade”. Essa mudança de contexto não costuma causar falhas de página (*page faults*).

(v) Copy-on-Write (COW) permite que processos pais e filhos compartilhem inicialmente as mesmas páginas na memória. Se qualquer um dos processos modifica a página compartilhada, somente nesse momento a página é copiada.

9. (1,0) Sobre gerenciamento de armazenamento, julgue os seguintes itens como verdadeiro (V) ou falso (F).

(v) Escalonamento de disco diz respeito a estratégias empregadas pelo SO para minimizar o tempo de busca por dados e aumentar a largura de banda de disco, mediante o emprego de vários algoritmos para escalonar o atendimento a pedidos de I/O de disco, regulando as movimentações do cabeçote de leitura.

(f) Algoritmos como LOOK e C-LOOK consideram a movimentação até a extremidade do disco enquanto algoritmos como SCAN e C-SCAN consideram a movimentação apenas até a requisição mais externa em cada direção, possuindo o potencial para diminuir a quantidade de movimentações do cabeçote de leitura/escrita.

(v) Com relação ao uso de espaço de *swap* em armazenamento secundário, como extensão da memória principal, é correto afirmar que este se tornou menos comum devido a aumentos na capacidade de memória.

(f) Espaço de swap em armazenamento secundário não pode ser pego a partir do sistema de arquivo normal. Geralmente existe unicamente mediante o uso de uma partição de disco separada (*raw*).

(f) Estrutura de discos em RAID é primariamente usada para realizar o *backup* de arquivos em múltiplos discos. Preconiza o funcionamento de múltiplos discos de forma incremental (ex. 6 discos de 1 TB disponibilizam conjuntamente 6 TB permitindo um uso otimizado do espaço desses discos).

10. (1,0) Sobre gerenciamento de armazenamento, selecione a alternativa correta.

(x) RAID 0 contribui primariamente para o aumento da velocidade de leitura/escrita de informações em disco, tendo em vista que a informação é dividida em pedaços (*strips*), em múltiplos discos, de maneira não redundante.

() RAID 1, de forma similar ao RAID 0, também realiza a divisão da informação em múltiplos pedaços e as copia para o disco. Também não envolve redundância de informação.

() RAID 5 implementa um esquema de paridade para favorecer a recuperação de informações em caso de perda de um disco, porém, imprime como sobrecarga a delegação de um disco para o armazenamento das paridades.

() Todas as afirmações anteriores estão erradas.